



LUMINA JOIAS

Sistema de Gerenciamento de Loja de Joias

Documentação do Projeto

Análise e Projeto de Aplicações Web

Prof. Dr. Vitor Mesaque A. Lima

Versão 1.0 · 2025

1. Visão Geral do Sistema

1.1 Descrição

O Lumina Joias é uma aplicação web para gerenciamento completo de uma loja de joias, permitindo o controle de produtos, clientes, pedidos e funcionários. O sistema oferece uma experiência elegante e intuitiva, refletindo a sofisticação do segmento de joias.

1.2 Problema Resolvido

Lojas de joias frequentemente dependem de planilhas ou sistemas genéricos que não atendem às necessidades específicas do setor, como controle de materiais preciosos, variações de produto (tamanho, pedra, metal) e gestão de pedidos personalizados. O Lumina Joias resolve esses problemas com um sistema especializado.

1.3 Público-Alvo

- Proprietários e gestores de lojas de joias
- Vendedores e atendentes
- Clientes que desejam acompanhar pedidos e histórico de compras

2. Requisitos Funcionais

2.1 Autenticação e Controle de Acesso

ID	Descrição	Prioridade
RF-01	Cadastro de usuário com nome, e-mail e senha	Alta
RF-02	Login com e-mail e senha	Alta
RF-03	Logout seguro com invalidação de sessão	Alta
RF-04	Área protegida acessível apenas a usuários autenticados	Alta
RF-05	Perfis de acesso: Administrador e Vendedor	Média

2.2 Gestão de Produtos (Joias)

ID	Descrição	Prioridade
RF-06	Cadastrar joias com nome, descrição, material, pedra, peso e preço	Alta
RF-07	Listar joias com filtros por categoria e material	Alta

RF-08	Editar informações e preço de uma joia	Alta
RF-09	Excluir joia do catálogo (soft delete)	Alta
RF-10	Upload de imagem do produto	Média
RF-11	Controle de estoque por unidade	Alta

2.3 Gestão de Clientes

ID	Descrição	Prioridade
RF-12	Cadastrar cliente com nome, CPF, e-mail, telefone e endereço	Alta
RF-13	Listar clientes com busca por nome ou CPF	Alta
RF-14	Editar dados do cliente	Alta
RF-15	Excluir cliente (somente se não houver pedidos)	Alta
RF-16	Visualizar histórico de pedidos do cliente	Média

2.4 Gestão de Pedidos

ID	Descrição	Prioridade
RF-17	Criar pedido associado a um cliente com um ou mais produtos	Alta
RF-18	Listar pedidos com filtro por status e data	Alta
RF-19	Atualizar status do pedido (Pendente, Confirmado, Entregue, Cancelado)	Alta
RF-20	Visualizar detalhes completos de um pedido	Alta
RF-21	Cancelar pedido com justificativa	Média

2.5 Dashboard e Relatórios

ID	Descrição	Prioridade
RF-22	Dashboard com total de pedidos, faturamento e estoque baixo	Alta
RF-23	Gráfico de vendas por período	Média
RF-24	Lista dos produtos mais vendidos	Média

3. Requisitos Não Funcionais

ID	Categoria	Descrição
RNF-01	Segurança	Senhas armazenadas com hash bcrypt
RNF-02	Segurança	Autenticação via JWT com expiração configurável
RNF-03	Usabilidade	Interface responsiva para desktop e mobile
RNF-04	Desempenho	Respostas da API em menos de 500ms
RNF-05	Manutenibilidade	Código organizado com padrão MVC e separação de camadas
RNF-06	Disponibilidade	Sistema disponível via deploy em nuvem (Render ou Railway)
RNF-07	Validação	Todas as entradas validadas no frontend e no backend

4. Modelo de Dados

4.1 Entidades Principais

O sistema possui quatro entidades centrais, descritas abaixo com seus atributos e relacionamentos.

4.1.1 Usuário (users)

Campo	Tipo	Restrição	Descrição
id	UUID	PK, NOT NULL	Identificador único
name	VARCHAR(100)	NOT NULL	Nome completo
email	VARCHAR(150)	UNIQUE, NOT NULL	E-mail de login
password_hash	VARCHAR(255)	NOT NULL	Senha criptografada
role	ENUM	DEFAULT 'seller'	admin seller
created_at	TIMESTAMP	DEFAULT NOW()	Data de criação

4.1.2 Produto / Joia (products)

Campo	Tipo	Restrição	Descrição
id	UUID	PK, NOT NULL	Identificador único
name	VARCHAR(150)	NOT NULL	Nome da joia
description	TEXT	NULLABLE	Descrição detalhada

material	VARCHAR(50)	NOT NULL	Ex: Ouro 18k, Prata 925
gemstone	VARCHAR(50)	NULLABLE	Ex: Diamante, Esmeralda
weight_g	DECIMAL(8,2)	NOT NULL	Peso em gramas
price	DECIMAL(10,2)	NOT NULL	Preço de venda
stock	INTEGER	DEFAULT 0	Quantidade em estoque
image_url	VARCHAR(255)	NULLABLE	URL da imagem
deleted_at	TIMESTAMP	NULLABLE	Soft delete

4.1.3 Cliente (customers)

Campo	Tipo	Restrição	Descrição
id	UUID	PK, NOT NULL	Identificador único
name	VARCHAR(100)	NOT NULL	Nome completo
cpf	CHAR(11)	UNIQUE, NOT NULL	CPF sem formatação
email	VARCHAR(150)	UNIQUE, NOT NULL	E-mail de contato
phone	VARCHAR(20)	NULLABLE	Telefone/WhatsApp
address	TEXT	NULLABLE	Endereço completo
created_at	TIMESTAMP	DEFAULT NOW()	Data de cadastro

4.1.4 Pedido (orders)

Campo	Tipo	Restrição	Descrição
id	UUID	PK, NOT NULL	Identificador único
customer_id	UUID	FK customers	Cliente do pedido
user_id	UUID	FK users	Vendedor responsável
status	ENUM	DEFAULT 'pending'	pending confirmed delivered cancelled
total	DECIMAL(10,2)	NOT NULL	Valor total do pedido
notes	TEXT	NULLABLE	Observações
created_at	TIMESTAMP	DEFAULT NOW()	Data do pedido

4.1.5 Item do Pedido (order_items)

Campo	Tipo	Restrição	Descrição
id	UUID	PK, NOT NULL	Identificador único
order_id	UUID	FK orders	Pedido relacionado
product_id	UUID	FK products	Produto vendido
quantity	INTEGER	NOT NULL	Quantidade

unit_price	DECIMAL(10,2)	NOT NULL	Preço no momento da venda
------------	---------------	----------	---------------------------

4.2 Relacionamentos

- Um Usuário pode criar vários Pedidos (1:N)
- Um Cliente pode ter vários Pedidos (1:N)
- Um Pedido contém um ou mais Itens de Pedido (1:N)
- Um Produto pode estar em vários Itens de Pedido (1:N)

5. Arquitetura da Aplicação

5.1 Stack Tecnológica

Camada	Tecnologia	Justificativa
Frontend	React + Tailwind CSS	SPA moderna com design responsivo e eficiente
Backend	NestJS (Node.js)	Framework robusto, modular e com TypeScript nativo
Banco de Dados	PostgreSQL	Banco relacional confiável, suportado no Railway/Render
ORM	Prisma ORM	Migrations automáticas e tipagem segura com TypeScript
Autenticação	JWT + bcrypt	Padrão de mercado para APIs REST stateless
Versionamento	Git + GitHub	Histórico, branches por feature e colaboração em equipe
Deploy Backend	Render / Railway	Plataformas gratuitas com suporte a Node.js e PostgreSQL
Deploy Frontend	Vercel	Deploy gratuito e otimizado para React

5.2 Arquitetura em Camadas (NestJS)

- Controller: recebe as requisições HTTP e valida os DTOs
- Service: contém a lógica de negócio
- Repository (Prisma): abstrai o acesso ao banco de dados
- Guard: verifica autenticação JWT em rotas protegidas
- Interceptor: padroniza as respostas da API

5.3 Principais Endpoints da API REST

Método	Rota	Descrição
POST	/auth/register	Cadastrar novo usuário
POST	/auth/login	Autenticar e receber JWT
GET	/products	Listar produtos com filtros
POST	/products	Cadastrar novo produto
PATCH	/products/:id	Editar produto
DELETE	/products/:id	Soft delete do produto
GET	/customers	Listar clientes
POST	/customers	Cadastrar cliente
PATCH	/customers/:id	Editar cliente
DELETE	/customers/:id	Remover cliente
GET	/orders	Listar pedidos com filtro de status
POST	/orders	Criar pedido com itens
PATCH	/orders/:id/status	Atualizar status do pedido
GET	/dashboard	Dados para o painel principal

6. Validações do Sistema

6.1 Validações de Formulário (Frontend + Backend)

Campo	Regra	Mensagem de Erro
E-mail	Formato válido (RFC 5322)	"Informe um e-mail válido"
Senha	Mínimo 8 caracteres	"A senha deve ter ao menos 8 caracteres"
CPF	11 dígitos com dígito verificador	"CPF inválido"
Preço	Valor positivo > 0	"O preço deve ser maior que zero"
Estoque	Inteiro >= 0	"Estoque não pode ser negativo"
Peso	Decimal > 0	"Informe um peso válido"
Nome	Não vazio, mínimo 2 chars	"O nome é obrigatório"

7. Versionamento e Organização Git

7.1 Estrutura de Branches

Branch	Finalidade
main	Código estável e pronto para produção
develop	Integração das features antes de ir para main
feature/auth	Implementação de login, cadastro e JWT
feature/products	CRUD completo de produtos/joias
feature/customers	CRUD completo de clientes
feature/orders	Criação e gestão de pedidos
feature/dashboard	Painel de métricas e gráficos
fix/validations	Correção de bugs nas validações

7.2 Estrutura do Repositório

- lumina-joias/
 - o | backend/ (NestJS API)
 - o | | src/
 - | | | auth/
 - | | | products/
 - | | | customers/
 - | | | orders/
 - | | | dashboard/
 - o | | prisma/ (schema e migrations)
 - o | | frontend/ (React + Tailwind)
 - o | | src/
 - | | | pages/
 - | | | components/
 - | | | services/ (chamadas à API)

8. Telas da Aplicação

#	Tela	Descrição
1	Login	Formulário de e-mail e senha com validação
2	Cadastro de Usuário	Criação de conta com confirmação de senha
3	Dashboard	KPIs: total de pedidos, faturamento, estoque baixo, gráfico de vendas
4	Lista de Produtos	Tabela com filtros e ações de editar/excluir
5	Cadastro de Produto	Formulário com todos os campos da joia

6	Detalhes do Produto	Visualização completa com imagem e histórico
7	Lista de Clientes	Busca por nome/CPF, listagem paginada
8	Cadastro de Cliente	Formulário com dados pessoais e endereço
9	Lista de Pedidos	Filtro por status e data, badge de status colorido
10	Novo Pedido	Seleção de cliente, adição de produtos ao carrinho
11	Detalhes do Pedido	Itens, valor total, status e botão de atualizar

9. Deploy e Instruções de Execução

9.1 Execução Local

Pré-requisitos: Node.js 18+, npm, PostgreSQL rodando localmente.

Backend (NestJS)

1. Clonar o repositório e entrar em /backend
2. Copiar .env.example para .env e preencher DATABASE_URL e JWT_SECRET
3. Executar: npm install
4. Executar: npx prisma migrate dev
5. Executar: npm run start:dev
6. API disponível em http://localhost:3000

Frontend (React)

7. Entrar em /frontend
8. Copiar .env.example e definir VITE_API_URL=http://localhost:3000
9. Executar: npm install
10. Executar: npm run dev
11. Aplicação disponível em http://localhost:5173

9.2 Deploy em Produção

- Backend: Railway (com PostgreSQL integrado) ou Render
- Frontend: Vercel (conectar repositório GitHub, auto-deploy no push para main)
- Configurar variáveis de ambiente na plataforma escolhida